



Primer Parcial

Cloud Provider Analytics

Juana Mendonca (61185), Sol Rodriguez(63029) y María Agustina Sanguinetti
(63115)

Big Data - Proyecto de Curso

4 de Mayo de 2026

Índice

1. Diagrama de Arquitectura de Alto Nivel	3
1.1. Patrón Elegido: Lambda	3
1.2. Arquitectura y Diagrama	3
2. Mapeo de Requisitos a Componentes	5
2.1. Asignación de Componentes y Justificación Técnica	5
2.2. Abordaje de las 5Vs del Big Data aplicado al caso:	7
3. Flujo de Datos	7
3.1. Entrada de datos	9
3.2. Flujo batch	9
3.2.1. Bronze	9
3.2.2. Silver	9
3.2.3. Gold	9
3.3. Flujo streaming	10
3.3.1. Bronze	10
3.3.2. Silver	10
3.3.3. Gold	11
3.4. Publicación en AstraDB	11
4. Asunciones y Riesgos Iniciales	12
4.1. Asunciones	12
4.2. Riesgos	12
5. Estimación de esfuerzo y recursos	13
5.1. Distribución de tareas	13
5.2. Estimación de esfuerzo	13
5.3. Recursos	14

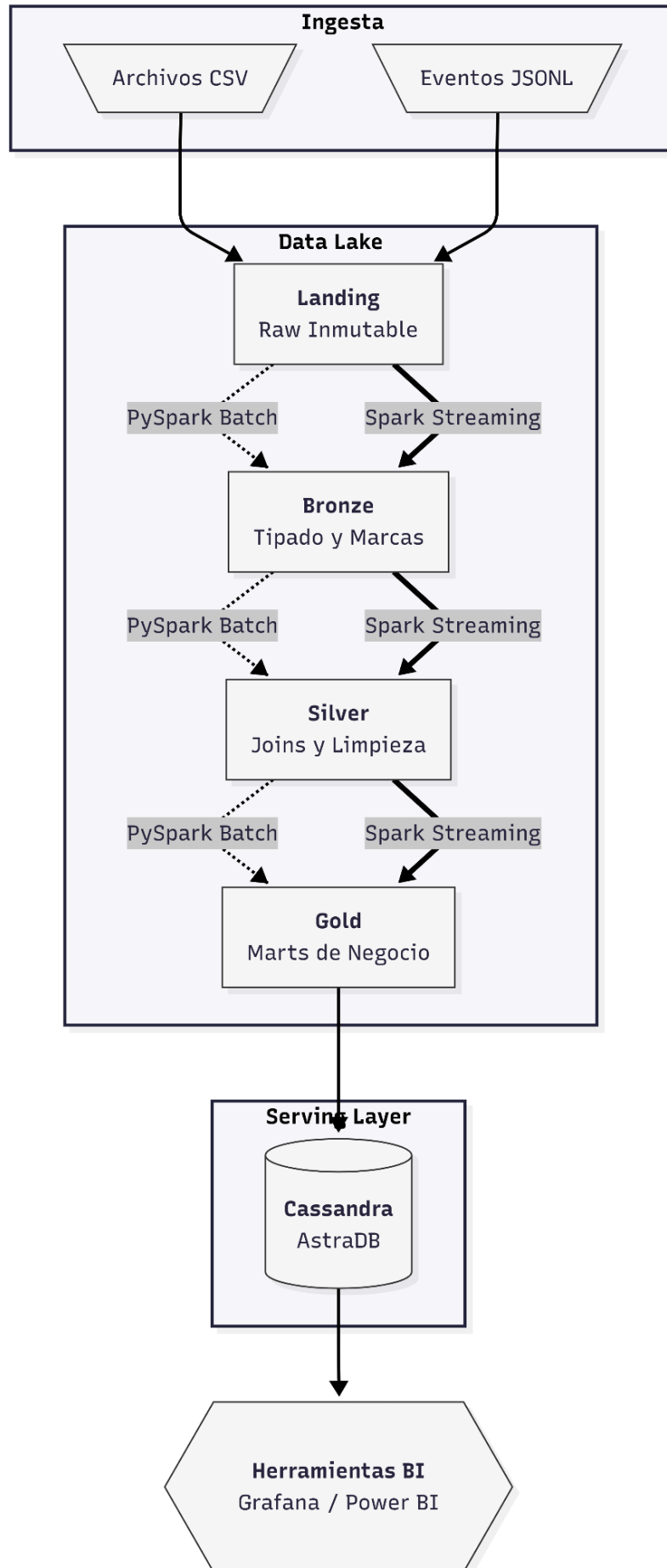
1. Diagrama de Arquitectura de Alto Nivel

1.1. Patrón Elegido: Lambda

El proyecto requiere procesar métricas operativas en *near real-time* y procesar datos maestros y facturación en modo *batch*. Lambda permite separar ambos flujos. Se descartó Kappa porque procesar archivos históricos estáticos (CSV) como si fueran *streams* añade complejidad innecesaria para el alcance y las herramientas del proyecto.

1.2. Arquitectura y Diagrama

- **Ingesta (Fuentes):** Los datos crudos (CSV y JSONL) tal cual llegan.
- **Procesamiento:** PySpark (batch) y Spark Structured Streaming (streaming).
- **Almacenamiento (Zonas del Data Lake):**
 - **Landing (Raw Immutable):** Almacenamiento de los archivos originales sin ninguna modificación.
 - **Bronze (Raw Estándar):** Datos convertidos a formato Parquet con tipificación explícita. Se aplica deduplicación por `event_id` y se agregan marcas de auditoría (`ingest_ts`, `source_file`).
 - **Silver (Conformado):** Capa de limpieza y cruces. Se normalizan campos, se resuelven los *joins* con dimensiones, se tratan nulos y *outliers*, y se controla la evolución de esquemas.
 - **Gold (Marts de negocio):** Vistas analíticas específicas para FinOps, Soporte y Producto, listas para ser consumidas.
- **Serving Layer:** Base de datos NoSQL (AstraDB/Cassandra) donde se publican los marts de la capa Gold. La carga se realiza mediante Spark Cassandra Connector para el flujo batch y mediante `foreachBatch` con driver Python para el flujo streaming.



2. Mapeo de Requisitos a Componentes

2.1. Asignación de Componentes y Justificación Técnica

Requisito Técnico / Negocio	Componente / Tecnología Asignada	Justificación y Aplicación en el Dataset
Ingesta batch de maestros y facturación	PySpark Batch	Lectura programada de los 7 archivos estáticos CSV que no requieren actualización segundo a segundo.
Streaming near real-time de eventos	Spark Structured Streaming	Consumo continuo de los archivos events_part_*.jsonl procesados en micro-lotes para actualizar consumos y costos.
Almacenamiento escalable y económico	Data Lake con Parquet Particionado	Almacenamiento dividido en zonas (Bronze/Silver/Gold). Se organizan en carpetas (particiones) por fecha o servicio para optimizar el motor de consultas.
Manejo de schema evolution (v1/v2)	schema explícito	La evolución de esquema (v1/v2) se gestiona mediante la definición de un esquema explícito que contempla tanto los campos originales como los nuevos atributos introducidos en versiones posteriores. De esta forma, los eventos de versiones anteriores presentan valores nulos en los nuevos campos, permitiendo mantener un modelo unificado sin depender de inferencia automática de esquema. Se evita el uso de <code>mergeSchema</code> para garantizar mayor control sobre la estructura de los datos, mayor velocidad y evitar inconsistencias en el procesamiento, especialmente en el flujo streaming.

Calidad de datos y cuarentena	Reglas lógicas (Capa Silver)	Aísla registros rotos o problemáticos (ej. campos vacíos en la columna credits de facturación). Los envía a una carpeta de cuarentena para no frenar el proceso y permitir su posterior reparación.
Detección de anomalías	Filtros estadísticos (Capa Silver)	identifica y frena valores que matemáticamente son válidos pero irreales para el negocio (ej. picos absurdos o negativos en la columna cost_usd_increment del stream).
Enriquecimiento y joins	Stream-Static Join (Capa Silver)	Cruce de la columna org_id del flujo continuo de JSONL con la dimensión estática customers_orgs.csv para obtener industria y plan.
Marts de FinOps, Soporte y Producto	Vistas y Agregaciones (Capa Gold)	Tablas pre-calculadas. Ej: Unificación de ticket_id y sla_breached para soporte; cálculo de subtotal + taxes para revenue mensual.
Serving para dashboards (BI)	AstraDB (Cassandra)	Base de datos NoSQL. Recibe la exportación de la capa Gold modelada en colecciones específicas para ser consultadas por Grafana/PowerBI.
Idempotencia y reprocesamiento	dropDuplicates en Spark	Uso de event_id como clave única temporal para evitar cobrarle dos veces el mismo evento al cliente si el pipeline se reinicia.
Gobierno y trazabilidad	Metadatos en Capa Bronze	Agregado automático de columnas de auditoría como ingest_ts (timestamp de procesamiento) y source_file (nombre exacto del CSV/JSONL de origen).

2.2. Abordaje de las 5Vs del Big Data aplicado al caso:

- **Volumen:** El sistema está diseñado para procesar grandes volúmenes de datos, incluyendo múltiples archivos (120 JSONL de eventos y 7 catálogos CSV). Se utiliza Spark como motor distribuido para escalar horizontalmente el procesamiento, mientras que el formato columnar Parquet permite reducir el tamaño de almacenamiento mediante compresión. Además, el particionado de datos facilita el acceso eficiente a subconjuntos relevantes, permitiendo escalar el pipeline a medida que crece el volumen de información.
- **Velocidad:** Resuelta mediante la Arquitectura Lambda. Las tablas maestras estáticas (usuarios, tickets) se procesan con latencia alta (Batch diario/mensual), mientras que el consumo operativo continuo (usage_events_stream) fluye en baja latencia (*near real-time*).
- **Variedad:** Se integran fuentes heterogéneas, incluyendo datos estructurados (CSV) y semi-estructurados (JSONL con estructuras anidadas), unificándolos en un modelo columnar común (Parquet). Además, se maneja la evolución de esquema (v1/v2) en el flujo de eventos, permitiendo incorporar nuevos atributos sin romper el pipeline.
- **Veracidad:** Asegurada mediante la deduplicación por event_id (idempotencia), la resolución de valores nulos o outliers, y el desvío a una zona de cuarentena de aquellos tickets inconsistentes o consumos financieramente anómalos.
- **Valor:** Entrega directa de los requerimientos analíticos del negocio. Se disponibilizan tableros accionables (evolución de tickets, facturación mensual normalizada a USD, etc.) publicados en *AstraDB*, consumibles con *Grafana* o *PowerBI*, listos para la toma de decisiones.

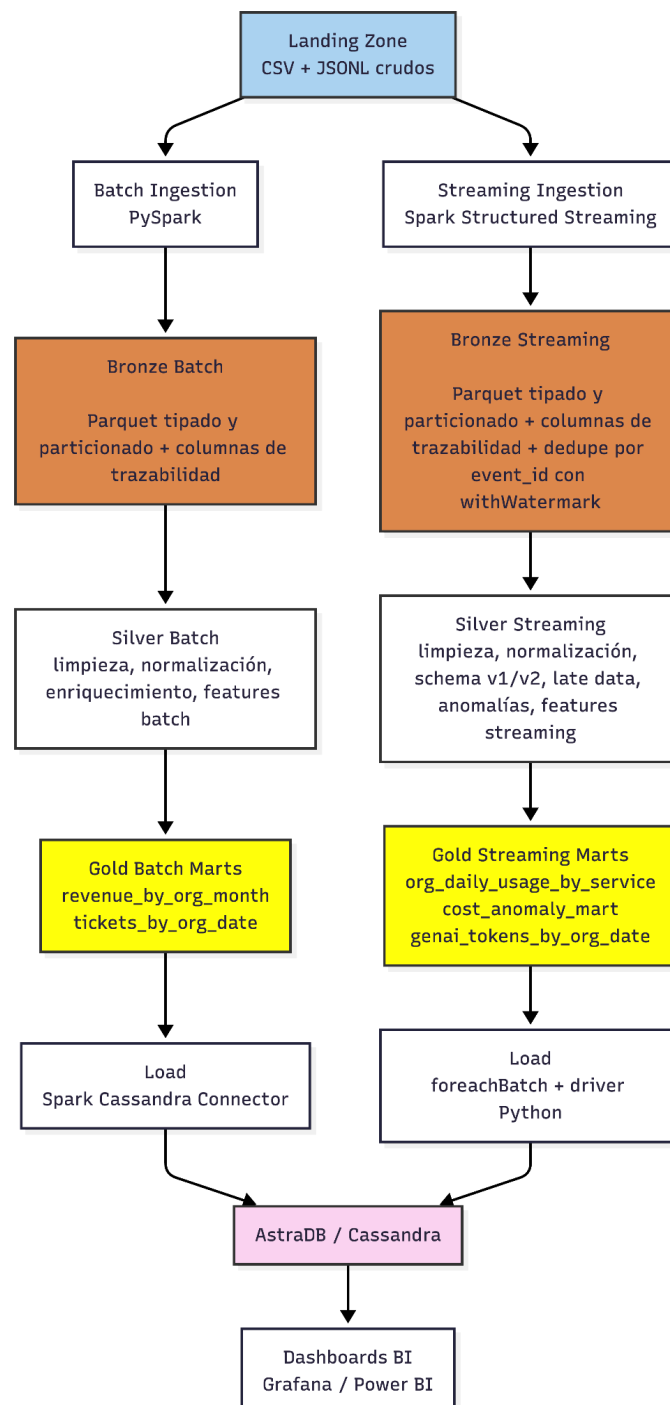
3. Flujo de Datos

El pipeline se divide en dos caminos principales, siguiendo el patrón Lambda definido previamente:

- un flujo batch para datos maestros, facturación, soporte, marketing y encuestas
- un flujo streaming para eventos de uso operativos.

Esta separación responde a que el proyecto exige procesamiento near real-time para métricas operativas como consumo y costos incrementales, pero procesamiento batch diario o mensual para CRM y facturación.

A continuación se exponen ambos flujos:



3.1. Entrada de datos

Los datos ingresan inicialmente en la capa **Landing**, donde se conservan sin modificaciones. En esta zona se reciben archivos CSV como `customers_orgs.csv`, `users.csv`, `resources.csv`, `support_tickets.csv`, `marketing_touches.csv`, `nps_surveys.csv` y `billing_monthly.csv`, además del stream de eventos `usage_events_stream/*`.jsonl

3.2. Flujo batch

3.2.1. Bronze

El flujo batch procesa los archivos estáticos mediante **PySpark Batch**. Primero lee los CSV desde la capa Landing utilizando esquemas explícitos, y los convierte a formato Parquet en la capa **Bronze**, aplicando particionado (por ejemplo por fecha). Durante esta etapa se agregan columnas de trazabilidad como `ingest_ts` y `source_file`. Esta capa mantiene el mismo grano que la fuente, sin agregaciones de negocio.

3.2.2. Silver

Luego, en esta etapa se aplican reglas de limpieza y normalización: casteo de tipos, tratamiento de nulos y outliers, normalización de fechas, regiones y servicios, y joins con dimensiones como organizaciones, usuarios y recursos. Los registros inválidos no bloquean el pipeline, sino que se separan en una zona de **quarantine** para revisión posterior.

Luego, los datos válidos son usados para calcular features como costos diarios. Al detectar valores anómalos mediante métodos estadísticos como z-score, MAD o percentiles, se generan flags que permiten identificarlos sin excluirlos del dataset.

Esta capa reduce el ruido, agrega contexto de negocio y deja los datos preparados para construir los marts Gold.

3.2.3. Gold

Finalmente, en **Gold** se generan marts batch como `revenue_by_org_month` y `tickets_by_org_date`. En esta etapa no se realizan transformaciones complejas, sino que se estructuran los datos según patrones de acceso (query-first), optimizando su almacenamiento y posterior consulta en sistemas como Cassandra.

Para cargar los datos en AstraDB se utiliza el **Spark Cassandra Connector**, ya que los marts en la capa Gold se generan como datasets consolidados sobre períodos completos (por ejemplo, mensual o diario). Esto permite realizar escrituras directas desde Spark a Cassandra sin necesidad de lógica adicional por micro-lote, simplificando la implementación y aprovechando la integración nativa del conector para cargas batch eficientes.

3.3. Flujo streaming

3.3.1. Bronze

El flujo streaming procesa los eventos JSONL mediante **Spark Structured Streaming**. En la capa **Bronze** se leen los eventos con esquema explícito, se agregan columnas de trazabilidad (`ingest_ts` y `source_file`), y se realiza deduplicación basada en `event_id`, utilizando `withWatermark` sobre el campo `timestamp` (para limitar el estado mantenido por Spark), garantizando idempotencia ante reprocesos o micro-lotes repetidos.

Los datos se almacenan en formato Parquet particionado por fecha. Esta decisión beneficia directamente a los esquemas de la capa Gold (como `org_daily_usage_by_service`, `cost_anomaly_mart` o `genai_tokens_by_org_date`), ya que se construyen sobre agregaciones diarias o rangos temporales. De esta forma, es posible leer únicamente las particiones relevantes, reduciendo el volumen procesado.

3.3.2. Silver

En Silver Streaming se compatibiliza la evolución de esquema entre versión 1 y versión 2 utilizando el campo `schema_version`. En Bronze, los campos introducidos en v2 como `carbon_kg` y `genai_tokens` se encuentran en `null` para eventos v1. En Silver, se define un modelo común manteniendo estas columnas y, según el caso, imputando valores por defecto para facilitar las agregaciones en Gold.

Por otro lado, en esta capa se consideran los efectos de datos tardíos (late data), identificados mediante el uso de `withWatermark` sobre el campo temporal del evento. Este mecanismo define un umbral de tolerancia al atraso, permitiendo incluir eventos dentro de ese margen en las agregaciones y excluyendo aquellos que exceden dicho límite, asegurando consistencia temporal en las métricas. Se opta por aplicar esta lógica en la capa Silver, ya que es donde se realizan las operaciones stateful (como agregaciones y ventanas).

Además, al igual que en la capa Silver de batch, se validan reglas de calidad como costos negativos fuera de tolerancia y campos nulos, derivando registros inválidos al parquet de quarantine.

En cuanto a valores atípicos, sobre métricas relevantes como `cost_usd_increment` se aplican métodos estadísticos (z-score, MAD o percentiles) para detectar anomalías, generando flags que se incorporan como features adicionales del dataset.

Además, una vez filtrados los registros inválidos, los eventos válidos se enriquecen mediante joins con dimensiones estáticas. En particular, se realiza un *stream-static join* entre el flujo de eventos y `customers_orgs` utilizando `org_id`, permitiendo incorporar atributos de negocio como industria, región y plan.

Finalmente, a partir de los datos validados y enriquecidos, se calculan las features pertinentes.

3.3.3. Gold

En Gold Streaming se publican datasets derivados de la capa Silver, organizados como marts orientados a las consultas de negocio. Estos incluyen vistas/tablas como `org_daily_usage_by_service`, `cost_anomaly_mart` y `genai_tokens_by_org_date`, diseñadas para responder eficientemente a queries analíticas.

3.4. Publicación en AstraDB

Los marts Gold se publican en **AstraDB/Cassandra**, modelando las tablas según las consultas esperadas. En vez de diseñar un modelo relacional general, se prioriza un diseño query-first, por ejemplo: costos diarios por organización y servicio, top servicios por costo, evolución de tickets críticos, revenue mensual y tokens GenAI diarios.

En cuanto a la carga de datos a Cassandra, se utiliza el `foreachBatch` con el driver de Cassandra porque los datos llegan en micro-lotes incrementales que pueden actualizar repetidamente las mismas claves de negocio. Esto permite controlar explícitamente los upserts, manejar reintentos sin duplicar métricas y definir la lógica de actualización ante reprocesos o errores parciales de escritura.

4. Asunciones y Riesgos Iniciales

4.1. Asunciones

- Se asume que los datos de entrada (CSV y JSONL) mantienen una estructura consistente dentro de cada versión de esquema (v1/v2), permitiendo su procesamiento mediante esquemas explícitos.
- Se considera que el volumen de datos es manejable dentro del entorno de ejecución (Google Colab), sin requerir infraestructura distribuida adicional.
- Se asume que el identificador `event_id` es único por evento, permitiendo aplicar deduplicación e idempotencia en el flujo streaming.
- Se considera que los datos batch (facturación, clientes, tickets) no requieren actualización en tiempo real y pueden procesarse en ventanas diarias o mensuales.
- Se asume que AstraDB puede manejar el volumen de escritura y consulta esperado para los marts generados.

4.2. Riesgos

- Datos inconsistentes o corruptos: presencia de valores nulos, tipos incorrectos o anomalías que puedan afectar cálculos.
 - a. *Mitigación:* validaciones en Silver y uso de zona de quarantine.
- Eventos duplicados en streaming: reintentos o reprocesamientos pueden generar duplicados.
 - a. *Mitigación:* deduplicación por `event_id` y uso de checkpointing.
- Evolución de esquema: incorporación de nuevos campos que no estén contemplados inicialmente.
 - a. *Mitigación:* uso de esquema explícito extensible con columnas opcionales (nullable), permitiendo compatibilizar versiones de datos (v1/v2) sin depender de inferencia automática.
- Datos tardíos (late data): eventos que llegan fuera de tiempo pueden afectar agregaciones.
 - a. *Mitigación:* uso de `withWatermark` y reglas de negocio para su inclusión/exclusión.
- Limitaciones de performance: operaciones costosas como joins o agregaciones pueden degradar el rendimiento.

- a. *Mitigación*: particionado adecuado, uso de Parquet y optimización de transformaciones.

5. Estimación de esfuerzo y recursos

5.1. Distribución de tareas

El desarrollo del pipeline se llevará a cabo por un equipo de tres integrantes, distribuyendo las tareas según los distintos componentes de la arquitectura.

- Integrante 1: Desarrollo del flujo batch (ingesta, Bronze, Silver, generación de marts Gold batch).
- Integrante 2: Desarrollo del flujo streaming (Structured Streaming, manejo de late data, deduplicación, generación de marts Gold streaming).
- Integrante 3: Modelado en Cassandra, carga de datos (batch y streaming) y definición de consultas para dashboards.

Adicionalmente, las tareas de documentación, validación de resultados y armado de la presentación serán realizadas de manera conjunta.

5.2. Estimación de esfuerzo

Se estima un esfuerzo total aproximado de **40 a 50 horas**, dividido en las siguientes fases:

Fase 1: Configuración e Ingesta (10 horas)

- Configuración del entorno Spark en Colab.
- Desarrollo de la ingesta Batch (CSV a Bronze) y Streaming (JSONL a Bronze).
- Implementación de particionado y metadatos.

Fase 2: Calidad de Datos y Capa Silver (15 horas)

- Desarrollo de reglas lógicas para nulos y anomalías (Cuarentena).
- Implementación de la evolución de esquema (`mergeSchema`)

Fase 3: Modelado Gold y Exportación (15 horas)

- Desarrollo de los Marts analíticos (FinOps, Soporte y Producto).
- Conexión mediante conector de Spark a AstraDB (Cassandra).
- Exportación exitosa de los datos finales.

Fase 4: Pruebas y Documentación (10 horas)

- Pruebas de estrés simulando caídas y verificando que no haya datos duplicados.
- Redacción del informe final y comentarios en el código.

5.3. Recursos

- Cómputo: Google Colab (Motor de Spark).
- Almacenamiento (Data Lake): Google Drive / Sistema de archivos local (Zonas Landing, Bronze, Silver, Gold en formato Parquet).
- Capa de Servicio: AstraDB (Cassandra) en su capa gratuita (*Free Tier*).